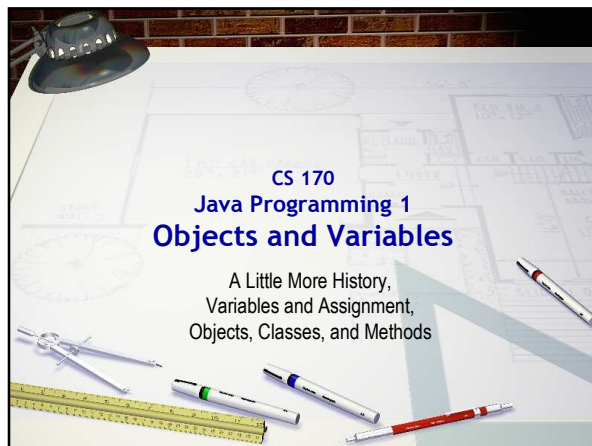
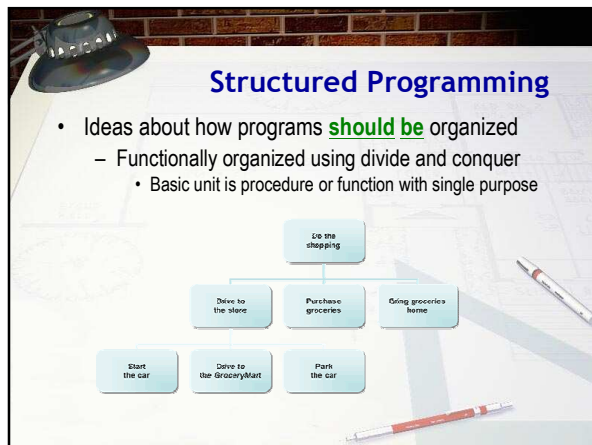






GOTO Considered Harmful

- Function should have single entrance, single exit
 - GOTO produced problem known as **spaghetti code**
 - Eliminated by application of sequence, selection, iteration
 - Boehm and Jacopini, Edsger Dijkstra

```
100 GOTO 500
110 PRINT I:
120 GOTO 400
130 PRINT I * 12:
140 GOTO 450
150 PRINT " = ";
160 GOTO 150
210 GOTO 150
240 PRINT " 12 ";
250 GOTO 200
400 PRINT " * ";
410 GOTO 300
450 I = I + 1
460 IF I > 12 THEN STOP
470 GOTO 110
500 PRINT "The Twelves Table"
510 I = 1
520 GOTO 110
```

Information Hiding

- With thousands of procedures, programs accidentally modified data used by another procedure
- Solution: "hide" data inside procedure: **local variables**
- Required a new type of language, called **block-structured** languages
- Pascal and C

```

int main()
{
    firstStep();
    secondStep();
}

void firstStep()
{
    int a = 10;
}

void secondStep()
{
    int b = 3;
}

```

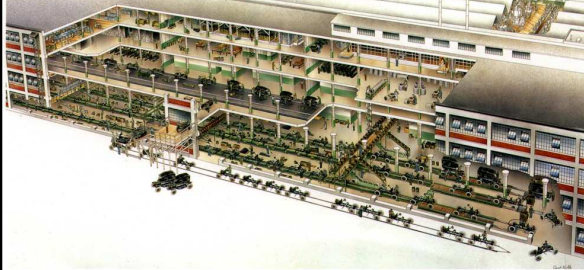
The main() function cannot see or change the variables a or b

The variable a is visible only inside the procedure firstStep(). It can be used and changed here but is invisible in all others.

The variable b can only be used inside of secondStep().

Structured Programming

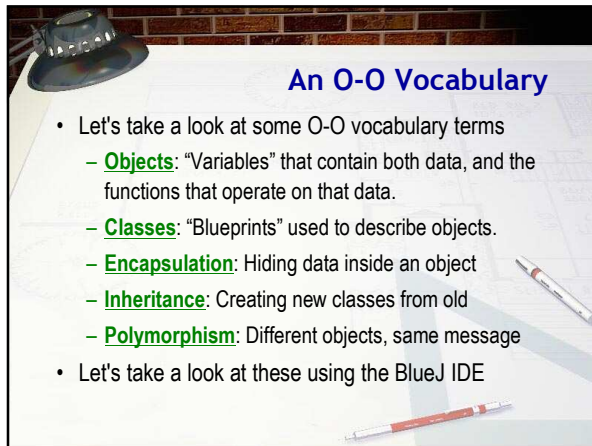
- Organized around a hierarchy of procedures
 - Effective for linear, assembly-line type problems



Object-Oriented Programming

- Better for building "reactive" software like GUIs
 - "Communities" with **objects** as basic building block
 - Self-contained components that work together
 - Windows, buttons, menus, scrollbars, etc.





An O-O Vocabulary

- Let's take a look at some O-O vocabulary terms
 - **Objects**: "Variables" that contain both data, and the functions that operate on that data.
 - **Classes**: "Blueprints" used to describe objects.
 - **Encapsulation**: Hiding data inside an object
 - **Inheritance**: Creating new classes from old
 - **Polymorphism**: Different objects, same message
- Let's take a look at these using the BlueJ IDE

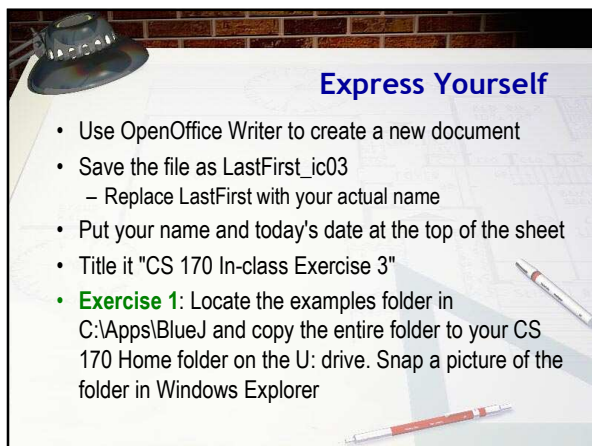


The BlueJ IDE

- A free Integrated Development Environment (IDE)
 - Written in Java, so it works on the Mac, Windows, Unix
 - Small
 - Must be installed
 - Integrated
 - Adds a
- Object-oriented
 - Interactive

BlueJ
University of Kent
Deakin University
University of Southern Denmark
Version 2.0.5

igger
ava snippets
ied
cts

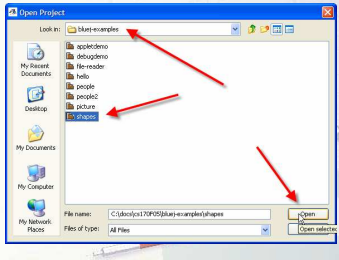


Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic03
 - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 3"
- **Exercise 1**: Locate the examples folder in C:\Apps\BlueJ and copy the entire folder to your CS 170 Home folder on the U: drive. Snap a picture of the folder in Windows Explorer

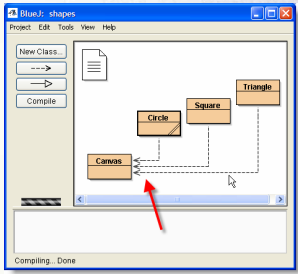
Examples

- Rename the folder to **bluej-examples**
 - So any changes you make will be there next time
- Choose **Open Project...** from the File menu
- Exercise 2:** Find and open the **Shapes** project and snap screen



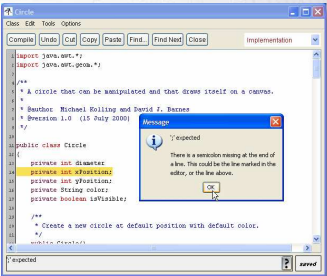
The Project Window

- Rectangle for each class
 - Diagonal lines mean "not compiled"
- Click **Compile** button
- Arrows represent class relationships
 - Circle**, **Square**, **Triangle** **use** the **Canvas** for display
- Open **editor** by double-clicking the **Circle** class



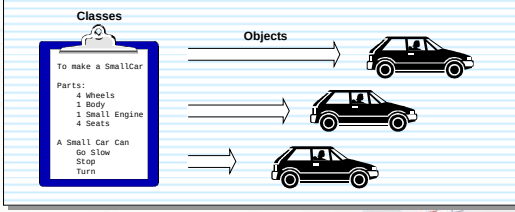
The Source Code

- Each source file gets an independent editor window
 - Remove semicolon after diameter and compile file
- Exercise 3:** snap
 - Error highlighted, simple message
 - Extended help
- Editing not "full-featured" as SciTE
- What are classes?



What Are Classes?

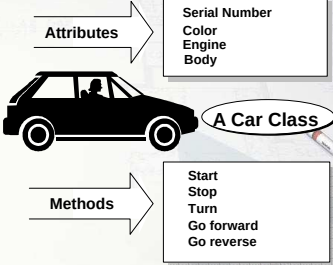
- A **pattern** or **blueprint** that describes the common characteristics of a category of objects



The diagram shows a clipboard titled 'Classes' with the following text: 'To make a SmallCar', 'Parts: 4 wheels, 1 body, 1 Small Engine, 4 seats', and 'A Small Car Can Go Slow, Stop, Turn'. Three arrows labeled 'Objects' point from the clipboard to three identical car icons.

The Class Definition

- Attributes
 - Object Data
- Methods
 - Object Actions



The diagram shows a car icon labeled 'A Car Class'. Two arrows point from the car to two boxes. The top box, labeled 'Attributes', contains 'Serial Number', 'Color', 'Engine', and 'Body'. The bottom box, labeled 'Methods', contains 'Start', 'Stop', 'Turn', 'Go forward', and 'Go reverse'.

Classes & Object Creation

- When you **define a class**, you specify
 - The data attributes that hold its object's state
 - The methods that define object behavior
- When you **create an object**
 - You create an **instance** of a class
 - These instances are called objects
 - This is also called **instantiation**
- You **send messages** to objects
 - Each message **invokes** a particular method



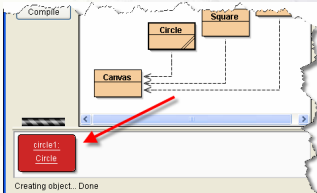
The diagram shows a car icon with a speech bubble saying 'I am a car!'.

What are Objects?

- **Components** used to build OO computer programs
 - User-defined variables used in OO programming
 - Represent visible objects, concepts, relationships
- All objects have three properties
 - **Identity**: who the object is
 - **State**: the object's characteristics
 - **Behavior**: what the object can do
- We'll use BlueJ to examine each of these properties

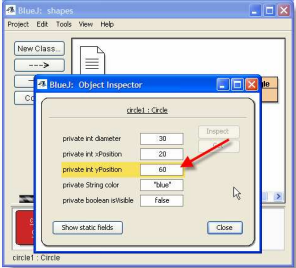
Creating Objects

- BlueJ allows you to **interactively** create objects and then send messages to those objects
- Let's create a few **Circle** objects
 - Right-click **Circle** class
 - Choose **new Circle()**
 - Name the **new instance circle1**
 - Object's **identity**
 - Create a 2nd **Circle**
- **Exercise 4**: snap



Object State

- **State**: **values** stored in an object's **data fields**
 - In BlueJ, we can inspect the object to examine its state
 - Right-click the **Circle** object named **circle1** on the Object Bench
 - Choose **Inspect**
- **Exercise 5**: snap
- Examine object's data
- Try changing one



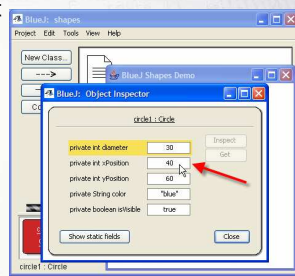
Encapsulation

- Three OOP principles used when writing programs
- Encapsulation**: objects responsible for their own data
 - Only access to object state is through public methods
- Similar to way that complex systems work in real world
 - Automobile: key, shift, accelerator, brake, steering
 - Computer: power button, mouse, keyboard
 - TV: remote control



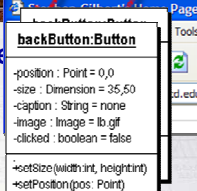

Object Behavior

- Behavior**: things the object can do (**methods**)
 - Right-click **circle1** and send the **makeVisible()** message to the object
- Choose **moveRight()** and then **Inspect**
 - Note how state can change in response to a message
- All **Circle** methods are **mutator** methods



Object Properties

- Identity**: the object's "name"
 - Your program can have several **Button** objects
 - Give each one a different name
- State**: **values** stored in an object's **data fields**
 - One **Button** object may be active, another disabled
 - One may use a left arrow, another, a right arrow
- Behavior**: things the object can do (**methods**)
 - A spreadsheet cell may "know" how to recalculate
 - A **Button** object can "be clicked"



Inheritance

- Create a new **class** using code in existing class
 - Newly created class is called a **subclass**
 - Think of it as a "child" class
 - The "parent" class is called the **superclass**
 - Subclass usually has new attributes or capabilities
 - "Inherits" attributes and capabilities of its parent

```
graph TD
    Button[Button] -- "Inherited Methods and Fields" --> ImageButton[ImageButton]
```

Polymorphism

- Means different subtypes respond to same message
 - General type responds to a particular method
 - Subclasses are responsible for their own actions
 - You don't know exactly "how" the action is carried out

```
graph TD
    Shape[Shape] --> Draw1((Draw))
    Shape --> Draw2[★ Draw]
    Shape --> Draw3[□ Draw]
```

Express Yourself

- **Exercise 6:** Use the shapes from the shapes project to create an image of a house and a sun. While you are doing this, write down the steps (in ic03) you took to make the picture. When done, snap a picture and place it below your list.
- **Exercise 7:** Close the shapes project and open the picture project. Try the **draw()** method. Also try the **setColor()** and **setBlackAndWhite()** methods. Shoot screenshots for each. How do you think the Picture program draws its shapes?

What are Variables?

- Programming languages store their data in **variables**
"Named, (usually changeable) storage area that holds a value"
- Like a "mailbox" or "bucket" that holds information



What are Values?

- The data stored inside a variable is called its **value**
 - A variable can have different values as a program runs
- Different **kinds** of values we can put in a variable
 - We can store different numbers **or** text in a variable

```
PI = 3.14159;  
myAge = 58;  
myStreetNumber = "575";
```

- PI holds a **real** number, **myAge** a **whole** number
- myStreetNumber** contains characters
 - Can't perform arithmetic on **myStreetNumber**

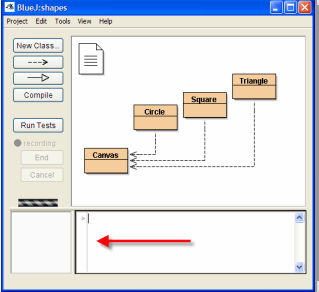
What are data types?

- Different **kinds** of variables for different **kinds** of data
- Different **sizes** of containers for **same kind** of data



Examining Types

- Use Code Pad to examine types and variables
- Choose **Show Code Pad** from View menu
- Expand the Code Pad area like this
- **Exercise 8:** snap a picture



Express Yourself

- Type your age, and press Enter
 - What do you see? What does that mean?
- Type your GPA (make sure you use a decimal point)
- Type your full name. What happens?
- Type your age and then + 5. What happens?
- Type your GPA and then * 2. What happens?
- Type your full name and then + 5. What happens?
- Type your full name and then * 5. What happens?
- **Exercise 9:** copy and paste the dialog from Code Pad

Creating Variables

- You can define (or create) variables in two places:
 - Inside a method : called a **local variable**
 - Outside a method : called a **field** or **instance variable**
- You define a local variable using this syntax:

type name = initial-value;

 - Parts in gray are optional.
 - The parts in red are required

Express Yourself

- Using Code Pad, create an appropriate variable for:
 - Your **age**
 - Your **gpa**
 - Your **name**
- **Exercise 10:** shoot a screen-shot of Code Pad
- In Code Pad, use **System.out.println()** to print your **age** variable to the console. (Remember you'll have to end the line with a semicolon.)
- **Exercise 11:** shoot a screen-shot of the result

Defining Local Variables

- So far, we've only learned about a few types
 - Type **int** stands for whole numbers
 - Type **double** stands for real numbers
 - Type **String** stands for a sequence of characters
- Here are some example declarations:

```
int x;  
double temperature;  
String college;
```

Placing Values in Variables

- Values are placed in variables using **assignment**
- Value can be from **input**, a **literal**, or an **expression**

```
int num = 23; // initializing assignment  
num = 45;    // executable assignment
```

```
num = readInt("Enter a whole number");  
PI = 3.14159;  
area = PI * radius * radius;
```

How Assignment Works

- Unlike a mailbox, a variable can hold **exactly one** value
 - Placing a value in a variable, (assignment), **replaces** the value previously stored there

```
myAge = 58;  
kathysAge = MyAge;  
kathysAge = 56;
```

- `kathysAge` contains 58, **then** 56
- `myAge` contains 58 for the length of the program

- We say that assignment performs a **destructive copy**


Assignment

- In Java, assignment is an operator, not statement
 - The job of the assignment operator is to:

Copy the value on its right into the variable on its left, and return the result copied as its value

- The variable on the left is changed when this happens
 - (Remember, this is a **destructive copy**)

Two Kinds of Assignment

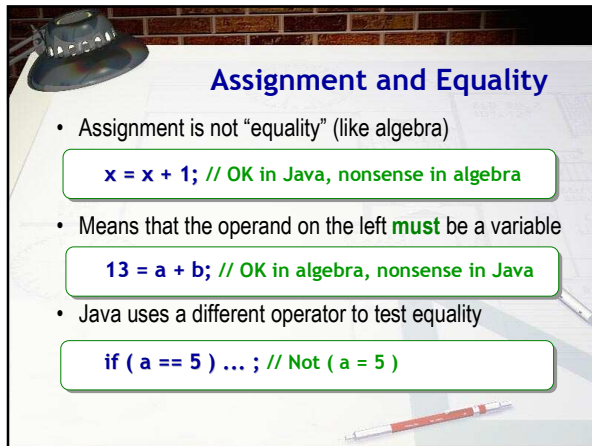
- The assignment operator is used in two situations
- 1) to give an initial value to a **new variable** at creation

```
int x = 5; // initialize i to 5
```

 - This is a **declaration**, **not** an executable statement
 - It can appear outside of any method
- 2) to copy a value into an **existing** variable

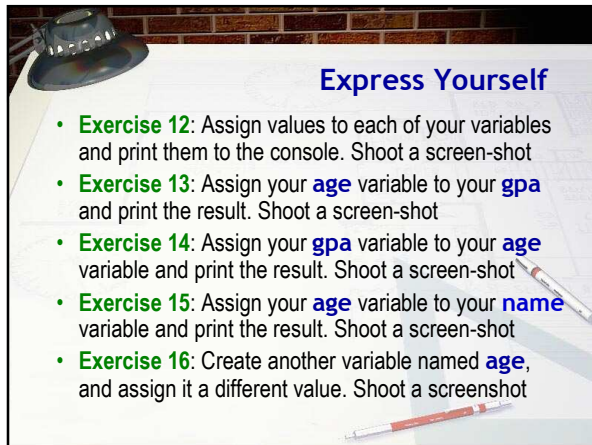
```
x = 23; // copy (store) the value 23 in location x
```

 - Executable statement; **must** appear inside a method



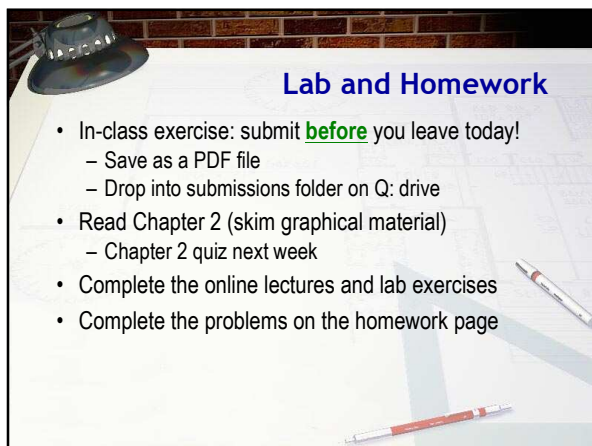
Assignment and Equality

- Assignment is not “equality” (like algebra)
`x = x + 1; // OK in Java, nonsense in algebra`
- Means that the operand on the left **must** be a variable
`13 = a + b; // OK in algebra, nonsense in Java`
- Java uses a different operator to test equality
`if ( a == 5 ) ... ; // Not ( a = 5 )`



Express Yourself

- **Exercise 12:** Assign values to each of your variables and print them to the console. Shoot a screen-shot
- **Exercise 13:** Assign your **age** variable to your **gpa** and print the result. Shoot a screen-shot
- **Exercise 14:** Assign your **gpa** variable to your **age** variable and print the result. Shoot a screen-shot
- **Exercise 15:** Assign your **age** variable to your **name** variable and print the result. Shoot a screen-shot
- **Exercise 16:** Create another variable named **age**, and assign it a different value. Shoot a screenshot



Lab and Homework

- In-class exercise: submit **before** you leave today!
 - Save as a PDF file
 - Drop into submissions folder on Q: drive
- Read Chapter 2 (skim graphical material)
 - Chapter 2 quiz next week
- Complete the online lectures and lab exercises
- Complete the problems on the homework page
